

# SOFTWARE REQUIREMENTS SPECIFICATION

## Advanced Web-Based Electronic Circuit Simulation

*A Browser-Native Interactive Circuit Simulator using Modified Nodal Analysis (MNA)*

Document Version: 1.0  
Prepared as per IEEE 830-1998 SRS Standard  
Date: 18 August 2026

|                   |                                                  |
|-------------------|--------------------------------------------------|
| Project Title     | Advanced Web-Based Electronic Circuit Simulation |
| Category          | Web-Based Engineering Simulation Tool            |
| Core Technologies | HTML5 Canvas, CSS3, Vanilla JavaScript           |
| Core Algorithms   | Modified Nodal Analysis, LU Decomposition        |
| Document Type     | Software Requirements Specification (SRS)        |
| Standard Followed | IEEE 830-1998                                    |

## Document Control

---

### Revision History

| Version | Date  | Description                                    | Author         |
|---------|-------|------------------------------------------------|----------------|
| 0.1     | Draft | Initial draft — Scope and Introduction         | Aawahan Bansal |
| 0.5     | Draft | Added Functional & Non-Functional Requirements | Aawahan Bansal |
| 1.0     | Final | Reviewed and finalized for submission          | Aawahan Bansal |

# Table of Contents

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                                                  | <b>5</b>  |
| 1.1 Purpose.....                                                              | 5         |
| 1.2 Document Conventions .....                                                | 5         |
| 1.3 Intended Audience and Reading Suggestions.....                            | 5         |
| 1.4 Project Scope .....                                                       | 5         |
| 1.5 Definitions, Acronyms, and Abbreviations .....                            | 6         |
| 1.6 References .....                                                          | 7         |
| <b>2. Overall Description .....</b>                                           | <b>8</b>  |
| 2.1 Product Perspective .....                                                 | 8         |
| 2.2 Product Functions (Summary) .....                                         | 8         |
| 2.3 User Classes and Characteristics.....                                     | 8         |
| 2.4 Operating Environment .....                                               | 8         |
| 2.5 Design and Implementation Constraints .....                               | 9         |
| 2.6 Assumptions and Dependencies.....                                         | 9         |
| <b>3. System Features (Functional Requirements) .....</b>                     | <b>10</b> |
| 3.1 Schematic Editor (Module: SCH).....                                       | 10        |
| 3.2 Component Library (Module: LIB) .....                                     | 10        |
| 3.3 Circuit Simulation Engine (Module: SIM) .....                             | 11        |
| 3.4 Real-Time Current Flow Visualization (Module: VIS) .....                  | 11        |
| 3.5 Virtual Oscilloscope (Module: OSC) .....                                  | 12        |
| 3.6 Project Persistence — Save / Load / Export (Module: PRJ) .....            | 12        |
| 3.7 Simulation Controls (Module: CTL) .....                                   | 12        |
| <b>4. External Interface Requirements .....</b>                               | <b>14</b> |
| 4.1 User Interfaces.....                                                      | 14        |
| 4.2 Hardware Interfaces .....                                                 | 14        |
| 4.3 Software Interfaces.....                                                  | 14        |
| 4.4 Communication Interfaces .....                                            | 15        |
| <b>5. System Architecture.....</b>                                            | <b>16</b> |
| 5.1 Architectural Overview .....                                              | 16        |
| 5.2 High-Level Component / Module Diagram (Description) .....                 | 16        |
| 5.3 Simulation Algorithm Workflow — MNA + LU Decomposition .....              | 17        |
| 5.4 Suggested Technology Stack.....                                           | 17        |
| <b>6. Data Requirements (Database / API Planning for Future Phases) .....</b> | <b>19</b> |
| 6.1 Future Conceptual Data Model .....                                        | 19        |
| 6.2 Future REST API Endpoints (Illustrative) .....                            | 19        |
| 6.3 Future Technology Recommendations.....                                    | 19        |
| <b>7. Non-Functional Requirements.....</b>                                    | <b>21</b> |
| 7.1 Performance Requirements.....                                             | 21        |
| 7.2 Usability Requirements.....                                               | 21        |
| 7.3 Reliability Requirements.....                                             | 21        |
| 7.4 Security Requirements.....                                                | 21        |
| 7.5 Maintainability Requirements .....                                        | 21        |
| 7.6 Portability Requirements.....                                             | 22        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 7.7 Scalability Requirements .....                                | 22        |
| <b>8. Use Case Descriptions .....</b>                             | <b>23</b> |
| 8.1 Use Case Summary .....                                        | 23        |
| 8.2 Detailed Use Case: UC-03 — Run DC Simulation .....            | 23        |
| 8.3 Detailed Use Case: UC-08 — Handle Invalid Circuit Error ..... | 23        |
| <b>9. System Testing Plan .....</b>                               | <b>25</b> |
| 9.1 Testing Approach .....                                        | 25        |
| 9.2 Sample Test Cases .....                                       | 25        |
| <b>10. Appendices .....</b>                                       | <b>26</b> |

# 1. Introduction

## 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, unambiguous, and detailed description of the requirements for the "Advanced Web-Based Electronic Circuit Simulation" system. This document defines the functional and non-functional requirements, system architecture, interface requirements, and constraints for the software, and is intended to serve as the primary reference for design, implementation, testing, and evaluation of the project.

This SRS is written for a final-year / semester college software engineering project and is intended to be realistic in scope for a small student development team (2-4 members) working over a single academic semester, while maintaining professional software engineering documentation standards.

## 1.2 Document Conventions

This document follows the IEEE 830-1998 recommended practice for software requirements specifications. Functional requirements are identified using the format FR-<Module>-<Number> (e.g., FR-SIM-01). Non-functional requirements are identified as NFR-<Number>. Priority levels used throughout this document are:

- High — Core requirement; the system is unusable or fails its primary purpose without it.
- Medium — Important requirement that significantly enhances functionality or usability.
- Low — Desirable feature that can be deferred to a later phase without affecting core deliverables.

## 1.3 Intended Audience and Reading Suggestions

| Audience                           | Purpose of Reading                                                                           |
|------------------------------------|----------------------------------------------------------------------------------------------|
| Faculty Evaluators / Project Guide | Assess project scope, feasibility, and technical depth against academic evaluation criteria. |
| Student Development Team           | Use as the primary reference for design, coding, module division, and sprint planning.       |
| UI/UX Designer (team member)       | Reference Section 4 (External Interface Requirements) for wireframes and interaction design. |
| Software Tester                    | Reference Section 3 (System Features) and Section 9 (Testing Plan) to design test cases.     |
| Future Contributors                | Reference Section 6 (Data Requirements) for backend/API/database extension in later phases.  |

## 1.4 Project Scope

The system is a fully client-side, browser-based electronic circuit simulator. It allows a user to visually construct analog electronic circuits using a drag-and-drop schematic editor, and simulate the electrical behaviour of the circuit (DC and transient analysis) using Modified Nodal Analysis (MNA) solved via LU Decomposition with partial pivoting. The simulation results are visualized in real time through animated current-flow indicators and a virtual oscilloscope.

In Scope:

- Interactive schematic editor with drag-and-drop placement of components on an HTML5 Canvas grid.
- A core component library: resistor, capacitor, inductor, DC voltage source, DC current source, ground, and connecting wires.
- Wire/node connection tool with automatic node detection and highlighting.
- A numerical simulation engine implementing MNA equation formulation and LU Decomposition with partial pivoting for solving the resulting linear systems.
- DC (steady-state) analysis and basic transient (time-domain) analysis for RC/RL circuits.
- Real-time animated current-flow visualization using moving dots along wires, with direction and relative speed indicating current magnitude and polarity.
- A virtual oscilloscope panel that plots voltage/current waveforms against time for user-selected probe points.
- Save/Load of circuit projects as local JSON files, and export of the schematic as an image (PNG).
- Undo/Redo, zoom/pan, grid-snap, and basic validation (e.g., floating nodes, short-circuit detection).

Out of Scope (for the current academic phase):

- Multi-user real-time collaboration and cloud account synchronization.
- Non-linear semiconductor device modelling (diodes, transistors, op-amps) beyond an optional stretch goal.
- PCB layout generation, SPICE-file import/export, and manufacturing/BOM features.
- Mobile native applications (the web app will be responsive but not published to app stores).

Future backend/API/database planning is documented in Section 6 for teams that wish to extend the project into a multi-phase or capstone project with persistent cloud storage and user accounts.

## 1.5 Definitions, Acronyms, and Abbreviations

| Term                   | Definition                                                                                                                                                                      |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MNA                    | Modified Nodal Analysis — a systematic method to form circuit equations ( $Ax = b$ ) from node voltages and branch currents of voltage sources/inductors.                       |
| LU Decomposition       | A numerical technique that factorizes a matrix $A$ into a Lower triangular matrix ( $L$ ) and an Upper triangular matrix ( $U$ ) to efficiently solve linear systems $Ax = b$ . |
| Partial Pivoting       | A numerical stability technique that reorders matrix rows during LU decomposition to place the largest available pivot element on the diagonal.                                 |
| Node                   | A point in the circuit where two or more component terminals are electrically connected.                                                                                        |
| Branch                 | A single circuit element connecting two nodes.                                                                                                                                  |
| Transient Analysis     | Time-domain simulation showing how voltages/currents evolve over time, essential for circuits with capacitors/inductors.                                                        |
| DC Operating Point     | The steady-state voltage/current values of a circuit under constant (non-time-varying) excitation.                                                                              |
| Oscilloscope (Virtual) | An in-app graphing panel that plots simulated signal values against time, emulating a physical oscilloscope.                                                                    |
| Canvas                 | The HTML5 <canvas> element used as the drawing surface for the schematic editor.                                                                                                |
| Netlist                | An internal data structure describing circuit components, their parameter values, and their node connections.                                                                   |

| Term  | Definition                           |
|-------|--------------------------------------|
| SRS   | Software Requirements Specification. |
| UI/UX | User Interface / User Experience.    |

## 1.6 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
2. Chua, L. O., Desoer, C. A., Kuh, E. S., "Linear and Nonlinear Circuits", McGraw-Hill.
3. Ho, C.W., Ruehli, A.E., Brennan, P.A., "The Modified Nodal Approach to Network Analysis", IEEE Transactions on Circuits and Systems, 1975.
4. Golub, G. H., Van Loan, C. F., "Matrix Computations", Johns Hopkins University Press — reference for LU Decomposition with partial pivoting.
5. MDN Web Docs — HTML5 Canvas API Reference ([developer.mozilla.org](https://developer.mozilla.org)).
6. ECMAScript (ES2020+) Language Specification — for Vanilla JavaScript implementation guidelines.

## 2. Overall Description

### 2.1 Product Perspective

The Advanced Web-Based Electronic Circuit Simulation system is a new, self-contained, client-side web application. It is not a modification of an existing system and has no dependency on any external circuit-simulation library — the numerical solver (MNA + LU Decomposition) is implemented entirely from scratch in Vanilla JavaScript, which is a core academic objective of the project (demonstrating applied numerical methods, not just UI construction).

The application runs entirely inside a modern web browser with no server-side computation required for the core simulation phase. This design choice keeps the project realistic for a student team: there is no backend infrastructure, authentication system, or database to maintain during the primary development phase, while Section 6 outlines an optional future architecture for teams that wish to extend the project with persistence and multi-device access.

### 2.2 Product Functions (Summary)

At a high level, the system provides the following major functions, each elaborated in detail in Section 3:

- Drag-and-drop schematic design of electronic circuits on a canvas-based grid.
- Automatic construction of a netlist (nodes, branches, component values) from the drawn schematic.
- Formulation of circuit equations using Modified Nodal Analysis.
- Solution of the resulting linear system using LU Decomposition with partial pivoting.
- Time-stepped transient simulation for circuits containing capacitors and/or inductors.
- Real-time animated visualization of current flow direction and magnitude using moving dots on wires.
- A virtual oscilloscope for plotting selected node voltages / branch currents over time.
- Local save/load of circuit designs and export of results/schematics.

### 2.3 User Classes and Characteristics

| User Class         | Description                                                                                                                         | Technical Expertise |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| Student / Learner  | Undergraduate students learning basic electronics/circuit theory who use the tool to build and visually understand simple circuits. | Low to Medium       |
| Educator / Faculty | Instructors who may use the tool to demonstrate circuit behaviour in a classroom setting or to evaluate the project.                | Medium              |
| Hobbyist Engineer  | Electronics hobbyists prototyping simple analog circuits before physical assembly.                                                  | Medium to High      |
| Project Evaluator  | Faculty panel assessing the correctness of simulation results and overall software engineering quality.                             | High                |

### 2.4 Operating Environment

The application is designed to operate under the following environment:

- Client: Any modern desktop web browser with HTML5 Canvas and ES2020+ JavaScript support — Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (latest two major versions).

- Operating System: Platform-independent (Windows, macOS, Linux) since the application runs entirely within the browser sandbox.
- Hosting: Static file hosting (e.g., GitHub Pages, Netlify, Vercel, or a local web server) is sufficient — no application server or database server is required for the core phase.
- Minimum Display Resolution: 1280 × 720 (responsive layout supports smaller laptop screens down to 1024 × 768).
- No installation is required; the application is accessed entirely through a URL.

## 2.5 Design and Implementation Constraints

- The simulation engine must be implemented in Vanilla JavaScript without external circuit-simulation or heavy linear-algebra libraries (e.g., no SPICE.js, no ngspice bindings), to fulfil the academic requirement of implementing MNA and LU Decomposition from first principles.
- Rendering must use the native HTML5 Canvas 2D API (no third-party rendering/game engines) to keep the project's graphics layer transparent and instructive.
- The project must run without requiring a backend server for its core functionality, since the student team does not have production server infrastructure.
- The numerical solver must handle matrices up to a realistic classroom-scale circuit size (a target of at least 30 nodes / branches) within real-time performance constraints defined in Section 7.
- The system must degrade gracefully (clear error messaging) for singular/ill-formed matrices such as floating nodes or circuits with no ground reference.

## 2.6 Assumptions and Dependencies

- Users have basic familiarity with fundamental circuit concepts (voltage, current, resistance, ground).
- The end-user's browser has JavaScript enabled and supports HTML5 Canvas.
- The development team has access to a modern code editor, Git/GitHub for version control, and a static hosting platform for deployment.
- Circuits simulated are linear, lumped-parameter analog circuits (resistors, capacitors, inductors, independent sources); non-linear device modelling is treated as an optional stretch goal, not a baseline dependency.

### 3. System Features (Functional Requirements)

This section describes each major feature of the system as a self-contained module, following the pattern: Description, Functional Requirements (with unique IDs and priority), and Processing Logic. Requirement IDs use the prefix FR followed by a module code.

#### 3.1 Schematic Editor (Module: SCH)

The schematic editor is the primary workspace where users visually construct circuits by placing components on a canvas-based, snap-to-grid surface and connecting them with wires.

| ID        | Requirement                                                                                                                                           | Priority |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-SCH-01 | The system shall render a canvas-based grid workspace that supports zooming and panning.                                                              | High     |
| FR-SCH-02 | The system shall allow users to drag components from a component palette and drop them onto the canvas.                                               | High     |
| FR-SCH-03 | The system shall snap placed components and wires to the nearest grid point to ensure clean alignment.                                                | High     |
| FR-SCH-04 | The system shall allow components to be selected, moved, rotated (90° steps), and deleted after placement.                                            | High     |
| FR-SCH-05 | The system shall allow the user to draw wires between component terminals by click-and-drag, automatically detecting connection points.               | High     |
| FR-SCH-06 | The system shall visually highlight connected nodes (e.g., a filled dot) to distinguish them from unconnected/floating terminals.                     | Medium   |
| FR-SCH-07 | The system shall support Undo and Redo for all schematic editing actions (place, move, delete, wire).                                                 | Medium   |
| FR-SCH-08 | The system shall allow the user to edit a component's properties (e.g., resistance value, capacitance, voltage) via a properties panel or dialog.     | High     |
| FR-SCH-09 | The system shall provide multiple selection (rubber-band/box select) and group delete/move.                                                           | Low      |
| FR-SCH-10 | The system shall validate the schematic before simulation and flag issues such as floating nodes, missing ground reference, or unconnected terminals. | High     |

#### 3.2 Component Library (Module: LIB)

A palette of standard, reusable circuit components forms the building blocks of any schematic.

| ID        | Requirement                                                                                                                                             | Priority |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-LIB-01 | The system shall provide the following base components: Resistor, Capacitor, Inductor, DC Voltage Source, DC Current Source, and Ground/Reference node. | High     |
| FR-LIB-02 | The system shall provide a Wire tool as a distinct drawing element connecting two node terminals.                                                       | High     |
| FR-LIB-03 | Each component shall have default parameter values (e.g., Resistor = 1 kΩ) which the user can edit.                                                     | High     |

| ID                  | Requirement                                                                                                            | Priority |
|---------------------|------------------------------------------------------------------------------------------------------------------------|----------|
| FR-LIB-04           | The system shall render distinct, standard schematic symbols (IEEE/IEC style) for each component type.                 | Medium   |
| FR-LIB-05 (Stretch) | The system may optionally provide an AC Voltage Source and a simple Diode as extended components in a later iteration. | Low      |

### 3.3 Circuit Simulation Engine (Module: SIM)

This is the computational core of the project. It converts the visually drawn schematic into a solvable mathematical model and computes the circuit's electrical behaviour.

| ID        | Requirement                                                                                                                                                                                                                   | Priority |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-SIM-01 | The system shall automatically generate a netlist (nodes, branches, component types, and values) from the schematic graph.                                                                                                    | High     |
| FR-SIM-02 | The system shall formulate the circuit equations using Modified Nodal Analysis (MNA), constructing the conductance matrix (A), the unknown vector (x: node voltages and source branch currents), and the source vector (b).   | High     |
| FR-SIM-03 | The system shall solve the linear system $Ax = b$ using LU Decomposition with partial pivoting, implemented natively in JavaScript.                                                                                           | High     |
| FR-SIM-04 | The system shall detect and report singular or ill-conditioned matrices (e.g., due to floating nodes) with a clear, user-readable error message instead of crashing.                                                          | High     |
| FR-SIM-05 | The system shall compute the DC operating point (steady-state node voltages and branch currents) for any valid resistive/source-only circuit.                                                                                 | High     |
| FR-SIM-06 | The system shall perform transient (time-domain) analysis for circuits containing capacitors and/or inductors, using companion models (e.g., Backward-Euler discretization) integrated into the MNA matrix at each time step. | High     |
| FR-SIM-07 | The system shall allow the user to configure simulation parameters: time step ( $\Delta t$ ) and total simulation duration.                                                                                                   | Medium   |
| FR-SIM-08 | The system shall re-run the simulation automatically whenever the circuit topology or a component value changes, provided the circuit is valid.                                                                               | Medium   |
| FR-SIM-09 | The system shall expose intermediate simulation data (per-node voltage, per-branch current, per-time-step) to the visualization and oscilloscope modules.                                                                     | High     |

### 3.4 Real-Time Current Flow Visualization (Module: VIS)

To make abstract simulation results intuitive, the system animates the flow of current directly on the schematic.

| ID        | Requirement                                                                                                                        | Priority |
|-----------|------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-VIS-01 | The system shall render animated yellow dots moving along each wire/branch to represent current flow.                              | High     |
| FR-VIS-02 | The direction of dot movement shall correspond to the actual (signed) direction of current flow computed by the simulation engine. | High     |
| FR-VIS-03 | The speed/density of the animated dots shall scale with the relative magnitude of the current in that branch.                      | Medium   |

| ID        | Requirement                                                                                                                            | Priority |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-VIS-04 | Branches carrying negligible or zero current shall show no dot animation (or a clearly reduced density) to avoid visual clutter.       | Medium   |
| FR-VIS-05 | The system shall allow the user to Play, Pause, and Reset the real-time animation independent of re-running the underlying simulation. | Medium   |
| FR-VIS-06 | The system shall display the computed voltage at each node and current through each branch as an optional on-canvas label/tooltip.     | Medium   |

### 3.5 Virtual Oscilloscope (Module: OSC)

The virtual oscilloscope provides a dedicated panel to plot signal values over time, similar to a physical bench oscilloscope.

| ID        | Requirement                                                                                                                                 | Priority |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-OSC-01 | The system shall allow the user to place one or more virtual probes on any node (voltage probe) or branch (current probe) in the schematic. | High     |
| FR-OSC-02 | The system shall render a time-domain waveform (voltage/current vs. time) for each active probe in a dedicated oscilloscope panel.          | High     |
| FR-OSC-03 | The oscilloscope shall support multiple simultaneous probe traces, each rendered in a distinct color with a legend.                         | Medium   |
| FR-OSC-04 | The system shall allow the user to adjust the oscilloscope's time-axis scale (time/div) and amplitude scale (volts or amps/div).            | Medium   |
| FR-OSC-05 | The system shall display numeric readouts (e.g., peak value, current cursor value) alongside the waveform display.                          | Low      |
| FR-OSC-06 | The system shall allow the user to export the oscilloscope plot as a PNG image or the underlying data as CSV.                               | Low      |

### 3.6 Project Persistence — Save / Load / Export (Module: PRJ)

| ID        | Requirement                                                                                                                                        | Priority |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-PRJ-01 | The system shall allow the user to save the current circuit design (components, positions, wiring, parameter values) as a JSON file to local disk. | High     |
| FR-PRJ-02 | The system shall allow the user to load a previously saved JSON project file and restore the full schematic state.                                 | High     |
| FR-PRJ-03 | The system shall allow the user to export the current schematic view as a PNG image.                                                               | Medium   |
| FR-PRJ-04 | The system shall auto-save the current project to browser local storage at regular intervals to reduce accidental data loss.                       | Low      |
| FR-PRJ-05 | The system shall provide a 'New Project' action that clears the canvas after user confirmation.                                                    | Medium   |

### 3.7 Simulation Controls (Module: CTL)

| ID        | Requirement                                                                                                              | Priority |
|-----------|--------------------------------------------------------------------------------------------------------------------------|----------|
| FR-CTL-01 | The system shall provide Run, Pause, and Stop/Reset controls for the simulation.                                         | High     |
| FR-CTL-02 | The system shall provide a Step control to advance a transient simulation by a single time step for detailed inspection. | Low      |
| FR-CTL-03 | The system shall display simulation status (Idle, Running, Error) clearly in the UI at all times.                        | Medium   |
| FR-CTL-04 | The system shall allow the user to select between DC (steady-state) and Transient simulation modes.                      | High     |

## 4. External Interface Requirements

### 4.1 User Interfaces

The application follows a single-page, three-panel layout designed for clarity and ease of use during both circuit design and simulation observation:

- Left Panel — Component Library: A scrollable palette of draggable component icons grouped by category (Passive, Sources, Tools).
- Center Panel — Schematic Canvas: The main HTML5 Canvas workspace with grid, zoom/pan controls, and a top toolbar (New, Save, Load, Undo/Redo, Run/Pause/Reset, DC/Transient toggle).
- Right Panel — Properties & Oscilloscope: A context-sensitive properties editor for the selected component, and a tabbed Oscilloscope view showing waveform plots for active probes.
- Status Bar (bottom): Displays simulation status, cursor coordinates, zoom level, and validation warnings/errors.

General UI/UX requirements:

- The interface shall use a clean, modern flat design with a consistent color palette and iconography.
- All interactive elements (buttons, draggable components) shall provide visual feedback on hover, drag, and selection states.
- Error and validation messages shall be shown as non-blocking toast notifications or inline banners, never as disruptive native browser alert() pop-ups.
- The layout shall be responsive down to a minimum resolution of 1024 × 768; panels may collapse into toggleable drawers below this width.
- Keyboard shortcuts shall be supported for common actions (Ctrl+Z / Ctrl+Y for undo/redo, Delete for removing a selection, Ctrl+S for save).

### 4.2 Hardware Interfaces

The system has no direct hardware interface requirements. It relies solely on standard input devices (mouse/trackpad and keyboard) supported natively by the host browser and operating system. No specialized peripherals, sensors, or external measurement instruments are required.

### 4.3 Software Interfaces

| Interface                                      | Description                                                                                                                                                |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Web Browser Runtime                            | The application executes within the browser's JavaScript engine (e.g., V8, SpiderMonkey) and renders via the HTML5 Canvas 2D rendering context.            |
| Browser Local Storage / File System Access API | Used for auto-save (local storage) and for save/load of JSON project files and PNG exports (file download/upload dialogs).                                 |
| Version Control (Development-time only)        | Git and a remote repository (e.g., GitHub) are used by the development team for source control; this is a development interface, not a runtime dependency. |
| Static Hosting Platform (Deployment-time only) | The compiled static assets (HTML/CSS/JS) are deployed to a static hosting service; no server-side runtime interface is required for the core application.  |

### 4.4 Communication Interfaces

For the core academic phase, the application performs no network communication after the initial page load — all computation (schematic editing, MNA formulation, LU decomposition, and rendering) executes locally in the browser. This eliminates network latency from the simulation loop and simplifies the project's deployment and evaluation. Section 6.2 describes an optional set of REST API communication interfaces for a future phase that introduces user accounts and cloud-saved projects.

## 5. System Architecture

### 5.1 Architectural Overview

The system follows a layered, client-only architecture. Each layer has a single, well-defined responsibility, which keeps the codebase modular, testable, and appropriate for division of work among student team members.

| Layer                   | Responsibility                                                                                                                              | Key Technology                               |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| Presentation / UI Layer | Renders the schematic canvas, component palette, toolbars, properties panel, and oscilloscope panel; captures user input (mouse, keyboard). | HTML5, CSS3, Canvas 2D API                   |
| Interaction Controller  | Translates raw UI events into schematic operations: place, move, wire, delete, select, undo/redo.                                           | Vanilla JavaScript (Event-driven)            |
| Circuit Model Layer     | Maintains the in-memory schematic graph — components, terminals, wires, and nodes — and generates the netlist.                              | JavaScript Data Structures (Graph)           |
| Simulation Engine Layer | Builds the MNA matrix/vector, performs LU Decomposition with partial pivoting, executes DC and transient solves.                            | Vanilla JavaScript (custom numerical module) |
| Visualization Layer     | Consumes simulation output to drive the animated current-flow dots and oscilloscope waveform rendering.                                     | Canvas 2D API, requestAnimationFrame         |
| Persistence Layer       | Serializes/deserializes circuit state to/from JSON; manages local storage auto-save and PNG export.                                         | Browser File API, LocalStorage API           |

### 5.2 High-Level Component / Module Diagram (Description)

Because this document is text-based, the architecture is described here as a data-flow narrative; the development team should render this as a block diagram in the design phase using a diagramming tool. Data flows as follows:

7. **User Interaction:** The user drags components from the Component Library onto the Schematic Canvas and connects them with wires.
8. **Circuit Model Update:** Every schematic edit updates the in-memory Circuit Graph (components, nodes, wires).
9. **Netlist Generation:** The Circuit Model Layer traverses the graph to produce a Netlist — a normalized list of components with their connected node indices and parameter values.
10. **MNA Formulation:** The Simulation Engine consumes the netlist to build matrix A and vector b according to Modified Nodal Analysis stamping rules.
11. **Linear Solve:** The engine performs LU Decomposition with partial pivoting on A, then forward/backward substitution to solve for x (node voltages and source currents).
12. **Transient Loop (if applicable):** For transient analysis, steps 4–5 repeat at each time step, updating capacitor/inductor companion models between iterations.
13. **Visualization:** The Visualization Layer reads the latest solution vector(s) each animation frame and updates dot positions/speeds on wires and oscilloscope trace buffers.
14. **Persistence:** On user request (or auto-save timer), the Circuit Model is serialized to JSON and written to a local file or browser storage.

### 5.3 Simulation Algorithm Workflow — Modified Nodal Analysis + LU Decomposition

This subsection details the core numerical workflow, which is the primary technical contribution of the project.

### Step 1 — Netlist Parsing and Node Numbering

All connected terminals are grouped into electrical nodes. One node (typically connected to the Ground/Reference component) is designated as node 0 (reference, 0 V). All remaining nodes are numbered 1..N for the purpose of matrix indexing.

### Step 2 — Matrix Sizing

For a circuit with N non-reference nodes and M independent voltage sources (and/or inductors treated via companion models requiring an auxiliary current variable), the MNA matrix A has dimensions  $(N+M) \times (N+M)$ , where the unknown vector x contains N node voltages followed by M branch currents.

### Step 3 — Stamping

Each component contributes ('stamps') fixed entries into A and b based on standard MNA stamping rules: resistors contribute conductance ( $1/R$ ) terms to the diagonal and off-diagonal entries of their connected nodes; independent sources contribute rows/columns for their auxiliary current variables; capacitors and inductors are converted into companion models (a resistor in parallel/series with a source, derived via Backward-Euler discretization) for transient analysis.

### Step 4 — LU Decomposition with Partial Pivoting

Matrix A is factorized into  $A = P \cdot L \cdot U$ , where L is unit lower-triangular, U is upper-triangular, and P is a row-permutation matrix chosen at each elimination step to select the largest-magnitude available pivot in the current column, ensuring numerical stability.

### Step 5 — Forward and Backward Substitution

Given L, U, and P, the system solves  $L \cdot y = P \cdot b$  via forward substitution, then  $U \cdot x = y$  via backward substitution, yielding the solution vector x.

### Step 6 — Result Mapping

The solved vector x is mapped back to physical quantities: node voltages are read directly; branch currents through resistors are derived using Ohm's Law from node voltage differences; branch currents through voltage sources/inductors are read directly from their auxiliary variables in x.

### Step 7 — Transient Time-Stepping (if applicable)

For transient analysis, steps 3–6 repeat for each discrete time step  $\Delta t$ , using the previous time step's capacitor voltage / inductor current to update the companion source terms, until the configured total simulation duration is reached.

## 5.4 Suggested Technology Stack

| Concern             | Recommended Technology     | Rationale                                                                                                         |
|---------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------|
| Structure & Styling | HTML5, CSS3 (Flexbox/Grid) | No framework dependency; keeps the project lightweight and fully aligned with the 'built from scratch' objective. |
| Rendering           | HTML5 Canvas 2D API        | Native browser support, full control over schematic and waveform                                                  |

| Concern                  | Recommended Technology                                          | Rationale                                                                                |
|--------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------|
|                          |                                                                 | drawing, good performance for 2D vector graphics.                                        |
| Application Logic        | Vanilla JavaScript (ES2020+), modular via ES Modules            | Meets the explicit 'built entirely from scratch' requirement; keeps bundle size minimal. |
| Numerical Engine         | Custom JavaScript modules for MNA assembly and LU Decomposition | Core academic deliverable — demonstrates understanding of numerical linear algebra.      |
| Build Tooling (optional) | Vite or plain ES Modules with a lightweight dev server          | Fast local development; optional bundling for production deployment.                     |
| Version Control          | Git + GitHub                                                    | Standard collaborative workflow for a student team.                                      |
| Deployment               | GitHub Pages / Netlify / Vercel (static hosting)                | Zero-cost, zero-maintenance hosting suitable for a client-only application.              |

## 6. Data Requirements (Database / API Planning for Future Phases)

The core academic deliverable is entirely client-side and requires no database. This section is provided for planning purposes only, so the project can be realistically extended in a future capstone phase to support user accounts, cloud-saved projects, and sharing — without requiring a redesign of the core simulation engine.

### 6.1 Future Conceptual Data Model

| Entity           | Key Attributes                                                                        | Description                                                                                                          |
|------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| User             | user_id (PK), name, email, password_hash, created_at                                  | A registered user account for cloud project storage.                                                                 |
| Project          | project_id (PK), user_id (FK), title, description, created_at, updated_at             | A saved circuit design owned by a user.                                                                              |
| CircuitSnapshot  | snapshot_id (PK), project_id (FK), json_payload, version, saved_at                    | A versioned JSON snapshot of a project's schematic state (components, wires, parameters), enabling history/rollback. |
| SimulationResult | result_id (PK), snapshot_id (FK), result_type (DC/Transient), data_blob, generated_at | Optionally cached simulation output for quick reload without recomputation.                                          |
| ShareLink        | share_id (PK), project_id (FK), access_token, permission, expires_at                  | An optional shareable read-only link to a project for classroom demonstration.                                       |

### 6.2 Future REST API Endpoints (Illustrative)

| Method | Endpoint                 | Purpose                                                |
|--------|--------------------------|--------------------------------------------------------|
| POST   | /api/auth/register       | Create a new user account.                             |
| POST   | /api/auth/login          | Authenticate a user and issue a session/JWT token.     |
| GET    | /api/projects            | List all projects owned by the authenticated user.     |
| POST   | /api/projects            | Create a new project (stores initial JSON schematic).  |
| GET    | /api/projects/{id}       | Retrieve a specific project's latest schematic JSON.   |
| PUT    | /api/projects/{id}       | Update/save a project's schematic JSON (new snapshot). |
| DELETE | /api/projects/{id}       | Delete a project.                                      |
| POST   | /api/projects/{id}/share | Generate a shareable read-only link for a project.     |

### 6.3 Future Technology Recommendations

- Database: A document-oriented store (e.g., MongoDB) fits naturally since project schematics are already JSON; alternatively PostgreSQL with a JSONB column for the schematic payload.
- Backend Runtime: Node.js with Express (or a lightweight framework) to reuse JavaScript across the stack, minimizing the learning curve for the same student team.
- Authentication: JWT-based stateless authentication for simplicity in a small-scale academic deployment.

- This future phase is explicitly optional and should only be pursued after the core simulation engine (Sections 3 and 5) is complete, tested, and stable.

## 7. Non-Functional Requirements

### 7.1 Performance Requirements

| ID     | Requirement                                                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | The system shall compute the DC operating point for a circuit of up to 30 nodes within 100 milliseconds on a typical laptop-class CPU.     |
| NFR-02 | The real-time current-flow animation shall render at a minimum of 30 frames per second for circuits of up to 50 components.                |
| NFR-03 | Transient simulation shall support at least 1,000 time steps without a perceptible (>1 second) delay in updating the oscilloscope display. |
| NFR-04 | The schematic editor shall respond to drag-and-drop and wiring actions with no perceptible input lag (<50 ms).                             |

### 7.2 Usability Requirements

- A first-time user shall be able to place a component, wire a simple series RC circuit, and run a DC simulation within 5 minutes without external documentation, aided by on-screen tooltips.
- All error messages (e.g., floating node, singular matrix) shall be phrased in plain, non-technical-jargon language with a suggested corrective action.
- The application shall provide an in-app quick-start guide or sample circuit templates to reduce onboarding friction.

### 7.3 Reliability Requirements

- The system shall not crash or freeze the browser tab when presented with an invalid or unsolvable circuit; it shall instead display a graceful error state.
- Auto-save (local storage) shall trigger at a maximum interval of 60 seconds during active editing to minimize data loss risk.
- The numerical solver shall be validated against known analytical solutions for standard textbook circuits (voltage dividers, RC charging curves) with a tolerance of less than 0.5% relative error.

### 7.4 Security Requirements

- Since the core application performs no network transmission of user data, the primary security concern is safe handling of local file import/export; the system shall validate and sanitize any loaded JSON project file before parsing to prevent malformed-data crashes.
- If the optional future cloud-backend phase (Section 6) is implemented, all authentication traffic shall use HTTPS, and passwords shall be stored using a salted hashing algorithm (e.g., bcrypt) — never in plain text.

### 7.5 Maintainability Requirements

- The codebase shall be organized into clearly separated modules corresponding to the architectural layers defined in Section 5.1 (UI, Circuit Model, Simulation Engine, Visualization, Persistence).
- The simulation engine (netlist builder, MNA stamper, LU solver) shall be implemented as pure, framework-independent JavaScript functions to allow independent unit testing.
- Code shall follow a consistent style guide (naming conventions, file structure) and include inline documentation for all public functions, particularly the numerical routines.

## **7.6 Portability Requirements**

- The application shall run correctly on the latest two major versions of Chrome, Firefox, Edge, and Safari without browser-specific code branches.
- The application shall require no OS-specific installation steps, functioning identically on Windows, macOS, and Linux via the browser.

## **7.7 Scalability Requirements**

- The architecture shall allow the component library (Section 3.2) to be extended with new component types (e.g., diodes, op-amps) without modifying the core MNA/LU engine, by adding new 'stamping' functions that conform to a common component interface.

## 8. Use Case Descriptions

This section describes the primary use cases of the system. A full UML use-case diagram should be developed by the team during the design phase using the actor and use-case list below; the two principal actors are the User (Student/Hobbyist) and, optionally in the future phase, an Educator/Admin actor.

### 8.1 Use Case Summary

| UC ID | Use Case Name                     | Primary Actor |
|-------|-----------------------------------|---------------|
| UC-01 | Build a Circuit Schematic         | User          |
| UC-02 | Edit Component Properties         | User          |
| UC-03 | Run DC Simulation                 | User          |
| UC-04 | Run Transient Simulation          | User          |
| UC-05 | Observe Real-Time Current Flow    | User          |
| UC-06 | Probe Signals on the Oscilloscope | User          |
| UC-07 | Save / Load Project               | User          |
| UC-08 | Handle Invalid Circuit Error      | User / System |

### 8.2 Detailed Use Case: UC-03 — Run DC Simulation

| Field          | Description                                                                                                                                                                                                                                                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actor          | User                                                                                                                                                                                                                                                                                                                                |
| Precondition   | A schematic with at least one valid closed loop and a ground reference has been drawn.                                                                                                                                                                                                                                              |
| Trigger        | User clicks the 'Run' button in DC mode.                                                                                                                                                                                                                                                                                            |
| Main Flow      | 1. System validates the schematic (FR-SCH-10). 2. System generates the netlist (FR-SIM-01). 3. System builds the MNA matrix (FR-SIM-02). 4. System solves via LU Decomposition (FR-SIM-03). 5. System maps results to node voltages/branch currents. 6. System updates the current-flow animation and enables oscilloscope probing. |
| Alternate Flow | If validation fails or the matrix is singular, the system displays an error message (see UC-08) and does not proceed to solving.                                                                                                                                                                                                    |
| Postcondition  | The circuit's DC operating point is available for visualization and probing.                                                                                                                                                                                                                                                        |

### 8.3 Detailed Use Case: UC-08 — Handle Invalid Circuit Error

| Field        | Description                                                                                      |
|--------------|--------------------------------------------------------------------------------------------------|
| Actor        | User / System                                                                                    |
| Precondition | User has requested a simulation run.                                                             |
| Trigger      | System detects a floating node, missing ground, or singular matrix during validation or solving. |

| Field         | Description                                                                                                                                                                                                                                                                                         |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Main Flow     | 1. System halts the simulation pipeline before/at the failure point. 2. System identifies the offending node(s)/component(s). 3. System highlights the issue visually on the canvas (e.g., red outline). 4. System displays a plain-language toast/banner explaining the issue and a suggested fix. |
| Postcondition | The user can correct the schematic and re-attempt simulation; no application crash occurs.                                                                                                                                                                                                          |

## 9. System Testing Plan

### 9.1 Testing Approach

Testing shall be conducted at three levels, appropriate for a college-scale project timeline:

- Unit Testing: Individual functions of the numerical engine (matrix stamping, LU decomposition, forward/backward substitution) shall be tested against known, hand-calculated or textbook results.
- Integration Testing: The flow from schematic edit → netlist generation → simulation → visualization shall be tested end-to-end using representative sample circuits.
- Usability/Acceptance Testing: A small group of test users (classmates) shall attempt standard tasks (build a voltage divider, observe an RC charging curve) and provide feedback against the criteria in Section 7.2.

### 9.2 Sample Test Cases

| Test ID | Description                                                                                                                         | Expected Result                                                                   |
|---------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| TC-01   | Solve a simple voltage-divider circuit (2 resistors + DC source) and compare computed node voltage to the analytical value.         | Computed voltage matches analytical value within 0.5% tolerance.                  |
| TC-02   | Simulate an RC charging circuit and compare the transient voltage curve to the analytical exponential charging formula.             | Simulated curve matches analytical curve within acceptable numerical tolerance.   |
| TC-03   | Attempt to simulate a circuit with a floating node (no ground reference).                                                           | System displays a clear validation error; no crash occurs.                        |
| TC-04   | Attempt to simulate a circuit with two independent voltage sources directly in parallel with conflicting values (over-constrained). | System detects the singular/conflicting matrix and reports an error.              |
| TC-05   | Draw, wire, and simulate a 20+ component circuit and verify UI responsiveness remains within NFR-01/NFR-04 thresholds.              | Simulation completes and UI remains responsive per performance targets.           |
| TC-06   | Save a project, reload the browser, and load the saved JSON file.                                                                   | The reloaded schematic is pixel-for-pixel/logically identical to the saved state. |
| TC-07   | Verify current-flow dot direction reverses correctly when a source polarity is flipped.                                             | Animated dots visibly reverse direction after re-simulation.                      |

## 10. Appendices

### Appendix A — Glossary

See Section 1.5 (Definitions, Acronyms, and Abbreviations) for the complete glossary of technical terms used throughout this document.

### Appendix B — Project Development Phasing (Recommended)

| Phase   | Focus                                                                                | Approx. Duration |
|---------|--------------------------------------------------------------------------------------|------------------|
| Phase 1 | Requirements finalization, UI wireframing, project setup, and canvas grid rendering. | 2 weeks          |
| Phase 2 | Schematic editor: drag-and-drop, wiring, component library, properties panel.        | 3 weeks          |
| Phase 3 | Simulation engine: netlist generation, MNA stamping, LU Decomposition, DC solve.     | 3 weeks          |
| Phase 4 | Transient analysis, current-flow animation, virtual oscilloscope.                    | 3 weeks          |
| Phase 5 | Save/Load, validation/error handling, UI polish, testing, and documentation.         | 2 weeks          |
| Phase 6 | Final testing, bug fixing, presentation preparation, and report writing.             | 1 week           |

### Appendix C — Assumptions Carried Forward

- All circuits under test are linear, time-invariant, lumped-element analog circuits.
- The student team has working knowledge of JavaScript and basic linear algebra (matrix operations) prior to starting Phase 3.
- Faculty evaluation will primarily assess correctness of simulation results, code quality/architecture, and UI/UX polish, consistent with the requirements in this document.

### Appendix D — Document Sign-Off

This Software Requirements Specification represents the agreed-upon scope for the Advanced Web-Based Electronic Circuit Simulation project as of the version and date noted on the cover page. Any changes to scope shall be recorded via an updated revision entry in the Document Control section.